

TASKVOICE — COLLEGE PROJECT VIVA & TECHNICAL PREPARATION GUIDE (HINGLISH VERSION)

Project Title: TaskVoice — Enterprise Voice-Powered Timesheet & Performance Management System

Target Audience: 4 Members ki Group

Course / Syllabus: Advanced Java Programming / Java EE (Servlets, JSP, JSTL, EJB, ORM, JPA)

Integrations: Google Gemini 1.5 Flash API, Aiven MySQL Cloud Database (Avian DB), Render Cloud Hosting via Docker.



TABLE OF CONTENTS

- [1. Executive Summary & Project Overview](#)
- [2. Group Member Work Division & Viva Roles \(4 Members\)](#)
- [3. Deep-Dive Technical Architecture & Integrations](#)
 - [- Google Gemini API Integration](#)
 - [- Aiven Cloud Database \(Avian DB / MySQL Cloud\)](#)
 - [- Render Cloud Hosting & Dockerization](#)
- [4. Syllabus Theory & Code Mapping \(Modules 1, 2, and 3\)](#)
 - [- Module 1: Java EE, Servlets, Cookies, Sessions & File Handling](#)
 - [- Module 2: JSP, Action Elements, EL & JSTL](#)
 - [- Module 3: EJB, ORM, JPA & DAO Architecture](#)
- [5. Exhaustive Viva Question & Answer Bank \(50+ Questions\)](#)
 - [- Section A: Servlets, Life Cycle & Web Tier Q&A](#)
 - [- Section B: JSP, JSTL, EL & State Management Q&A](#)
 - [- Section C: EJB, ORM, JPA & Data Architecture Q&A](#)
 - [- Section D: Gemini API, Aiven DB & Render Hosting Q&A](#)
 - [- Section E: Security, Testing & AI Usage Justification Q&A](#)

1. EXECUTIVE SUMMARY & PROJECT OVERVIEW

TaskVoice ek enterprise-grade Java EE web application hai jo manual timesheet tracking aur performance appraisals ko fully automate aur voice-power karti hai. Normally corporate me employees ko manually daily tasks aur hours type karne padte hain, lekin TaskVoice me employee bas mic icon daba kar naturally bolta hai aur AI automatically usko structured timesheet entries me convert kar deta hai.

Key Features:

1. **Voice-to-Timesheet Processing:** Browser me Web Speech API voice capture karti hai aur raw text backend me bhejti hai. Backend par Google Gemini 1.5 Flash API natural speech me se Project, Task, Duration (Hours), aur Description extract karke structured work blocks banata hai.
 2. **Data-Backed Performance Grading:** Managers completed tasks aur expected vs actual hours ke base par employees ko scores (\$0-100\$) aur letter grades (\$A+\$, \$A\$, \$B\$, \$C\$) dete hain.
 3. **AI-Assisted Appraisal Engine (Human-in-the-Loop):** Total logged hours, task completion rate, deadlines, aur manager remarks analyze karke rating aur increment percentage recommend karta hai. **Important Rule:** AI sirf *recommend* karta hai, final decision *Manager* ka hota hai.
 4. **Role-Based Access Control (RBAC):** `ADMIN`, `MANAGER`, aur `EMPLOYEE` teen alag roles ke liye strict URL security filters aur separate dashboards.
 5. **Production Infrastructure:** Project **Render Cloud** par Docker container me hosted hai, **Aiven Cloud MySQL Database** se connected hai, aur backend security keys environment variables se manage hoti hain.
-

2. GROUP MEMBER WORK DIVISION & VIVA ROLES (4 MEMBERS)

AI Tool Usage par Clear Clarification: External AI Agent (Google Gemini / Antigravity Agent) use karne ki permission college guidelines me mili thi. Viva me examiner ke samne AI usage ko **AI Pair Programmer** ki tarah present karna hai — yaani AI ne boilerplate code generation, Docker configuration, aur CSS design fast karne me help ki, jabki **Architecture, Database Schema, Filter Security Chain, Business Rules, aur End-to-End Logic aapki team ne design aur code kiya.**

Member	Designated Viva Role	Key Responsibilities & Module Focus	Key Code Artifacts
Member 1	Architecture, Core Servlet & Security Lead	- Servlet Lifecycle & Request Processing handling - Filter Security Chain (<code>AuthFilter</code> , <code>RoleFilter</code> , <code>CSRFFilter</code> , <code>EncodingFilter</code>) - Password Security (<code>JBCrypt</code> Hashing) & Session Management (<code>HttpSession</code>)	<code>AuthFilter.java</code> <code>RoleFilter.java</code> <code>LoginServlet.java</code> <code>PasswordUtil.java</code>
Member 2	Database Engineering & Aiven Cloud Lead	- Database Schema Design & Relational Constraints (<code>schema.sql</code> , <code>seed.sql</code>) - Apache DBCP2 Connection Pooling setup (<code>DBPoolListener</code>) - DAO (Data Access Object) pattern using PreparedStatements - Integration with Aiven Cloud Managed MySQL Database	<code>DBPoolListener.java</code> <code>UserDAOImpl.java</code> <code>TaskDAOImpl.java</code> <code>schema.sql</code>
Member 3	AI Integration & Gemini API Lead	- Browser Speech API interface (<code>voice-recorder.js</code>) <code>GeminiClient.java</code> (Java 17 <code>HttpClient</code> REST integration) - Prompt Engineering (<code>GeminiPrompts.java</code>) & Validation - Resilient Fallback Engine for API quota / offline handling	<code>GeminiClient.java</code> <code>GeminiService.java</code> <code>GeminiPrompts.java</code> <code>VoiceService.java</code>
Member 4	JSP/JSTL UI & Render Cloud Deployment Lead	- JSP Presentation Views, JSTL Core Tags (<code><c:forEach></code> , <code><c:if></code>), EL - File Handling & Dynamic PDF Export (<code>PdfBuilder</code> via Apache PDFBox) - Dockerization (<code>Dockerfile</code>) & Render Cloud Hosting (<code>render.yaml</code>)	<code>Dockerfile</code> <code>render.yaml</code> <code>PdfBuilder.java</code> <code>timesheet.jsp</code> , <code>dashboard.jsp</code>

3. DEEP-DIVE TECHNICAL ARCHITECTURE & INTEGRATIONS



A. Google Gemini 1.5 Flash API Integration

- **Purpose:** Natural Language Processing (NLP) - voice input parsing, progress update rephrasing, project sentiment analysis, aur appraisal score recommendations.
- **Implementation Detail:**
 1. No external heavy SDK — standard Java 17 `java.net.http.HttpClient` se direct REST integration kiya (`GeminiClient.java`).

2. HTTP `POST` request to `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=YOUR_API_KEY`.
3. **Jackson ObjectMapper** (`jackson-databind`) use karke prompts ko JSON serialize aur response ko JSON deserialize kiya.
4. `GeminiPrompts.java` me strict **Prompt Engineering** kiya taaki Gemini hamesha valid JSON format me hi reply kare.
5. **Resiliency & Fallback Engine**: Agar API key missing ho, internet down ho, ya rate limit (429) ho jaaye, toh `GeminiService.java` error catch karke internal mathematical fallback engine switch kar leta hai. **Application kabhi crash nahi hoti.**

B. Aiven Cloud Database (Avian DB / MySQL Cloud)

- **Purpose**: High-availability managed MySQL database jo Users, Projects, Tasks, Timesheets, Appraisals, aur Audit Logs ko securely store karta hai.
- **Implementation Detail**:
 1. Cloud-hosted MySQL 8.0 instance on **Aiven Cloud** (`taskvoice-db...aivencloud.com:26284`).
 2. **Connection Pooling**: App startup par `DBPoolListener.java` **Apache DBCP2** (`BasicDataSource`) initialize karta hai.
 3. DBCP2 max 20 connections open rakhta hai. Every request ke liye naya TCP connection open karne ka overhead khatam ho jata hai.
 4. Secured using SSL (`sslMode=REQUIRED`).
 5. Database secrets `.env` / Render Secrets me store kiye gaye hain.

C. Render Cloud Hosting & Dockerization

- **Purpose**: Cloud hosting aur automated container deployment.
 - **Implementation Detail**:
 1. **Dockerfile**: Multi-stage Docker build:
 - Stage 1: Build environment (JDK 17 + Maven) jo `taskvoice.war` package karta hai.
 - Stage 2: Runtime environment (Apache Tomcat 10 image `tomcat:10.1-jdk17-slim`).
 2. `render.yaml` : Infrastructure-as-Code (IaC):
 - Web service running on Docker.
 - Dynamic `$PORT` auto-binding.
 - Health check path `/health` mapped to `HealthCheckServlet.java` .
 - Secure environment variables binding (`DB_URL` , `DB_USERNAME` , `DB_PASSWORD` , `GEMINI_API_KEY` , `SESSION_SECRET`).
-

4. SYLLABUS THEORY & CODE MAPPING (MODULES 1, 2, AND 3)

MODULE 1: Java EE, Servlets, Cookies, Sessions & File Handling

1. What is Java EE / Enterprise Application?

- **Java EE (Enterprise Edition):** Ek standardized Java platform/specification hai jisme Servlets, JSP, JPA, EJB, JTA etc. shamil hain. Inka use large-scale, multi-tiered, secure aur scalable enterprise web applications banane me hota hai.
- **Tiered Architecture:**
 - *Client Tier:* Web browser jo HTTP requests bhejta hai.
 - *Web Tier:* Java Servlets, JSP, Filters jo UI logic aur routing handle karte hain.
 - *Business Tier:* Service classes jo business rules aur AI processing execute karti hain.
 - *Data Tier (EIS):* Cloud MySQL Database (Aiven DB) jo JDBC DAOs ke through access hota hai.

2. Servlet API & Life Cycle

- **Servlet:** Ek Java class hai jo server capabilities extend karti hai, specially HTTP protocol requests ko process karke dynamic response generate karne ke liye.
- **Servlet Life Cycle Methods:**
 1. `init(ServletConfig config)` : Tomcat jab servlet load karta hai, yeh method **sirf ek baar (once)** call hota hai initialization ke liye.
 2. `service(ServletRequest req, ServletResponse res)` : **Har HTTP request** par call hota hai. Default `HttpServlet.service()` implementation internal HTTP method check karke `doGet()`, `doPost()`, `doPut()`, `doDelete()` ko request pass karta hai.
 3. `destroy()` : Jab Tomcat server shutdown hota hai ya servlet unload hota hai, tab **sirf ek baar** cleanup ke liye call hota hai.
- **Deployment Descriptor (`web.xml`) vs Annotations (`@WebServlet`):**
 - Old Java EE me mapping `web.xml` file se hoti thi.
 - Java EE 6+ me annotations readable hote hain: `@WebServlet(name = "LoginServlet", urlPatterns = {"/login"})`. TaskVoice me humne `@WebServlet` aur `@WebFilter` use kiya hai.

3. RequestDispatcher: `forward()` vs `sendRedirect()`

Feature	<code>RequestDispatcher.forward()</code>	<code>HttpServletResponse.sendRedirect()</code>
Execution	Server-side internal transfer	Client-side HTTP 302 redirect
Network Overhead	1 HTTP Request & 1 Response	2 HTTP Requests & 2 Responses
Browser URL	URL change nahi hota	URL update ho kar naya page show karta hai
Request Scope	Request attributes (<code>req.setAttribute()</code>) preserve rehte hain	Request scope destroy ho jata hai (Naya request)
TaskVoice Code Example	<code>req.getRequestDispatcher("/WEB-INF/views/login.jsp").forward(req, res);</code>	<code>response.sendRedirect(request.getContextPath() + "/login");</code>

4. State Management: Cookies vs Session

- **Cookies:** Client-side browser me key-value pairs me store hoti hain.
- *Creation in Servlet:* `Cookie csrfCookie = new Cookie("COOKIE_CSRF_TOKEN", token); response.addCookie(csrfCookie);`
- *Security Flags:* `setHttpOnly(true)` (JavaScript access block karke XSS se bachata hai), `setSecure(true)` (Only HTTPS).
- **HttpSession:** Server-side memory me data store karta hai, jo client browser se `JSESSIONID` cookie ke through mapped hota hai.
- *Lifecycle:* `request.getSession(true)` se create hota hai, aur `session.invalidate()` (e.g. `LogoutServlet.java`) se destroy hota hai.
- *TaskVoice Usage:* Logged-in user data store karne ke liye (`SessionUtil.setUser(session, user)`).

5. File Handling in Servlets

- **File Uploading:** Servlet par `@MultipartConfig` annotation laga kar:
- `Part filePart = request.getPart("file");`
- `InputStream fileContent = filePart.getInputStream();`
- **File Downloading:** Response stream me binary data write karke:
- Headers set karna: `response.setContentType("application/pdf"); response.setHeader("Content-Disposition", "attachment; filename=\"report.pdf\")`;
- Data output stream me write karna: `ServletOutputStream out = response.getOutputStream();` (See `ReportServlet.java` & `PdfBuilder.java`).

MODULE 2: JSP, Action Elements, EL & JSTL

1. Introduction to JSP vs Servlets

- **Servlets:** Logic process karne me fast hain, par HTML render karna mushkil aur ganda hota hai (`out.println("<html>...");`).
- **JSP (Java Server Pages):** Text document jo static HTML ke saath dynamic Java tags incorporate karta hai. Presentation layer ke liye best hai.
- **JSP Execution / Life Cycle:**
 1. Translation: Tomcat `.jsp` file ko Java Servlet `.java` file me translate karta hai.
 2. Compilation: Tomcat `.java` ko bytecode `.class` me compile karta hai.
 3. Loading & Initialization: `jspInit()` method call hota hai.
 4. Execution: Har request par `_jspService(HttpServletRequest, HttpServletResponse)` call hota hai.
 5. Destruction: Unload par `jspDestroy()` call hota hai.

2. JSP Action Elements & Implicit Objects

- **Action Elements:** `<jsp:include page="..." />`, `<jsp:forward page="..." />`, `<jsp:useBean id="..." class="..." />`.
- **9 Implicit Objects:**
 1. `request` (`HttpServletRequest`)
 2. `response` (`HttpServletResponse`)
 3. `session` (`HttpSession`)
 4. `application` (`ServletContext`)
 5. `out` (`JspWriter`)
 6. `config` (`ServletConfig`)
 7. `pageContext` (`PageContext`)
 8. `page` (`Object` / `this`)
 9. `exception` (`Throwable` - sirf error pages me)

3. Expression Language (EL) & JSTL

- **Why JSTL fixes Scriptlet shortcomings:**
- Scriptlets (`<% Java code %>`) HTML me Java code mix kar dete hain, jo MVC pattern break karta hai, maintenance mushkil banata hai, aur XSS vulnerability create karta hai.
- **JSTL (Java Server Pages Standard Tag Library)** clean tags (`<c:forEach>`, `<c:if>`, `<c:choose>`) aur **EL** (`${user.name}`) provide karta hai jo code ko clean aur secure rakhte hain.
- **TaskVoice JSP Example:**

```
jsp <c:forEach items="${taskList}" var="task"> <tr> <td><c:out value="${task.title}"/></td> <td>
<c:if test="${task.status == 'COMPLETED'}"> <span class="badge bg-success">Completed</span> </c:if>
</td> </tr> </c:forEach>
```


MODULE 3: EJB, ORM, JPA & DAO Architecture

1. Enterprise JavaBeans (EJB) Architecture

- **EJB:** Java EE ka server-side software component model hai jo business logic, transactions, security, aur concurrency manage karta hai.
- **Types of Session Beans:**
- **Stateless Session Bean:** Specific client ka koi conversational state maintain nahi karta. Container inka pool maintain karta hai. High performance.
- **Stateful Session Bean:** Ek specific client ke saath multiple method calls ke dauran client state preserve rakhta hai.
- **Singleton Session Bean:** Puri application me ek hi single shared instance hota hai application-wide caching/state ke liye.

2. ORM & Java Persistence API (JPA)

- **Object-Relational Mapping (ORM):** Java objects (classes/POJOs) aur Relational Database tables (MySQL) ke beech mapping karne ki technique.
- **JPA Specifications:** Java EE ka standard specification (`jakarta.persistence`) jo `@Entity` , `@Table` , `@Id` , `@Column` annotations aur `EntityManager` interface through data persist karta hai.

3. Why TaskVoice Uses the DAO Pattern with DBCP2 Connection Pooling

- **Viva Defense Point:** Modern ORM frameworks (Hibernate/JPA) aur heavy EJBs startup memory aur execution overhead badha dete hain.
- **TaskVoice Architecture Choice:** Humne **DAO Pattern + Apache DBCP2 Connection Pooling + PreparedStatement** implement kiya.
- **Advantages:**
 1. *Zero Reflection Overhead:* Fast response time (< 20ms).
 2. *Full Query Control:* Timesheet aggregations aur reporting SQL queries fully optimized.
 3. *Security:* Parameterized `PreparedStatement` use karne se 100% SQL Injection Protection.
 4. *Resource Management:* `try-with-resources` se database connection leak zero.

5. EXHAUSTIVE VIVA QUESTION & ANSWER BANK (50+ QUESTIONS)

SECTION A: Servlets, Life Cycle & Web Tier Q&A

Q1: `GenericServlet` aur `HttpServlet` me kya difference hai?

Answer: `GenericServlet` ek protocol-independent servlet hai jo `Servlet` aur `ServletConfig` interfaces implement karta hai. `HttpServlet` `GenericServlet` ko extend karta hai specially HTTP protocol ke liye, aur HTTP specific methods provide karta hai jaise `doGet()` , `doPost()` , `doPut()` , `doDelete()` .

Q2: Servlet Life Cycle explain karo. `init()` method kitni baar call hota hai?

Answer: Servlet life cycle ke 3 main phases hote hain:

1. Initialization (`init()`): Tomcat jab servlet load karta hai, tab **sirf ek baar (exactly once)** call hota hai.
2. Execution (`service()`): **Har HTTP request** par call hota hai. Request receive karke `doGet()` ya `doPost()` ko dispatch karta hai.
3. Destruction (`destroy()`): Server shutdown ya servlet unload hone par **sirf ek baar** cleanup ke liye call hota hai.

Q3: Kya Servlets thread-safe hote hain? Tomcat concurrent requests kaise handle karta hai?

Answer: Servlets **inherently thread-safe nahi hote**. Tomcat har servlet ka **single instance** banata hai aur har incoming request ke liye ek naya worker thread create karta hai jo same servlet instance ke `service()` method ko call karta hai. Thread safety ke liye hum servlet ke instance variables me request data store nahi karte; hamesha method-local variables use karte hain.

Q4: TaskVoice me `forward()` aur `sendRedirect()` me kya difference hai?

Answer: `forward()` server-side internal transfer hai jisme browser ko pata nahi chalta (1 HTTP request, URL change nahi hota). `sendRedirect()` client-side HTTP 302 response bhejta hai jisse browser naya request bhejta hai (2 HTTP requests, URL change ho jata hai). TaskVoice me JSP render karne ke liye `forward()` aur POST form submit hone ke baad page reload issue se bachne ke liye `sendRedirect()` use karte hain.

Q5: TaskVoice me `ServletFilters` kaise kaam karte hain? Execution order kya hai?

Answer: Servlet Filters request ko servlets tak pahunchne se pehle aur response ko client tak lautne se pehle intercept karte hain. TaskVoice me chain execution order yeh hai:

1. `EncodingFilter` : UTF-8 character encoding set karta hai.
2. `CSRFFilter` : POST requests par CSRF token check karta hai.
3. `AuthFilter` : Session authentication check karta hai.
4. `RoleFilter` : User role (`ADMIN` , `MANAGER` , `EMPLOYEE`) permission check karta hai.

SECTION B: JSP, JSTL, EL & State Management Q&A**Q6: Scriptlets (`<% %>`) ke bajaye JSTL aur EL kyun use kiya?**

Answer: Scriptlets me HTML ke andar Java code likhna padta hai, jo MVC architecture break karta hai, maintenance mushkil banata hai, aur XSS attacks ke chances badha deta hai. **JSTL** tags (`<c:forEach>` , `<c:if>`) aur **EL** (`${user.name}`) clean, declarative markup provide karte hain jo code secure aur maintainable banate hain.

Q7: JSP ke 9 Implicit Objects ke naam batao. Sabse zyada konse use huye?

Answer: 9 implicit objects hain: `request`, `response`, `session`, `application`, `out`, `config`, `pageContext`, `page`, `exception`. Humne sabse zyada `request` (data pass karne ke liye), `session` (user login state ke liye), aur `application` (global context ke liye) use kiya.

Q8: JSP ke 4 Attribute Scopes kya hain?

Answer:

1. *Page Scope* (`pageContext`): Sirf current JSP page ke andar accessible.
2. *Request Scope* (`request`): Ek single HTTP request cycle (forwarded JSPs tak) me accessible.
3. *Session Scope* (`session`): Same user browser ke multiple requests me accessible.
4. *Application Scope* (`application`): Entire web application aur sabhi users ke liye accessible.

Q9: Agar browser me cookies disabled hon toh Session Tracking kaise hogi?

Answer: Session tracking browser ke `JSESSIONID` cookie par depend karti hai. Agar cookies disabled hon, toh **URL Rewriting** technique use hoti hai, jisme Tomcat har hyperlink URL ke aage `;jsessionid=XYZ123` append kar deta hai via `response.encodeURL(url)`.

Q10: Passwords ko database aur sessions me kaise secure kiya gaya hai?

Answer: Passwords plain text me kabhi store nahi hote. User creation aur password change par **JBCrypt** (`PasswordUtil.hash()`) se salt add karke hash generate kiya jata hai. Login ke waqt `PasswordUtil.check()` plain password ko hashed output se verify karta hai.

SECTION C: EJB, ORM, JPA & Data Architecture Q&A**Q11: Enterprise JavaBeans (EJB) kya hai? Session Beans ke types batao.**

Answer: EJB Java EE ka server-side component framework hai jo enterprise logic, transaction management, aur security handle karta hai. Session Beans ke 3 types hain:

- *Stateless Session Bean*: Client state store nahi karta; container pool maintain karta hai.
- *Stateful Session Bean*: Client session ke dauran multiple method calls tak state maintain karta hai.
- *Singleton Session Bean*: Application-wide single shared instance.

Q12: Object-Relational Mapping (ORM) aur JPA kya hai?

Answer: ORM ek technique hai jo Java classes/objects ko relational database tables me map karti hai. JPA Java EE ka standard API specification hai jo `@Entity`, `@Table`, `@Id` annotations aur `EntityManager` interface se persistence manage karta hai.

Q13: Java EE me JPA hone ke bavajood TaskVoice me DAO Pattern kyun use kiya?

Answer: TaskVoice ko lightweight, high-performance aur Render Cloud ke free tier par fast deployable rakhne ke liye humne **DAO Pattern + Apache DBCP2 Connection Pooling** choose kiya. Isse heavy JPA/Hibernate reflection startup overhead eliminate hota hai aur SQL query control milta hai.

Q14: Database Connection Pooling TaskVoice me kaise work karti hai?

Answer: Remote cloud database (Aiven DB) se baar-baar naya connection create karna slow (\$100-300\text{ms})\$) hota hai. `DBPoolListener.java` application startup par **Apache DBCP2** pool initialize kar deta hai. Jab bhi Servlet ko database operation karna hota hai, woh pool se open connection leta hai (`dataSource.getConnection()`) aur query finish hone par connection pool ko return kar deta hai (`close()`).

Q15: DAO Layer me SQL Injection se kaise bache?

Answer: Humne saari database queries me `PreparedStatement` with parameterized inputs (`?`) use kiye hain. MySQL driver user inputs ko compile time par execute nahi hone deta, jisse SQL Injection 100% prevent ho jata hai.

SECTION D: Gemini API, Aiven DB & Render Hosting Q&A

Q16: Google Gemini 1.5 Flash API TaskVoice me voice processing kaise karti hai?

Answer:

1. Client browser Web Speech API se voice record karke text bhejta hai.
2. `VoiceTimesheetServlet` raw speech string receive karta hai.
3. `GeminiService` `GeminiPrompts.java` se structured prompt generate karta hai.
4. `GeminiClient` Java `HttpClient` ke through Google Gemini 1.5 Flash REST endpoint par HTTP POST request bhejta hai.
5. Gemini JSON response bhejta hai jisme Project, Task, Duration aur Description formatted milta hai.

Q17: Gemini API Key kaha store hoti hai? Kya browser me leak hone ka risk hai?

Answer: `GEMINI_API_KEY` strictly server-side environment variables (`.env` locally, Render Secrets in production) me store hoti hai. Browser JS me kabhi nahi jaati. Saari API calls Java backend execute karta hai.

Q18: Demo ke waqt agar Gemini API down ho jaaye ya rate limit lage toh kya hoga?

Answer: `GeminiService.java` me **Data-Driven Fallback Engine** built-in hai. Agar Gemini API timeout, invalid key, ya 429 quota error de, toh system exception catch karke internal mathematical/rule-based calculation engine par switch kar leta hai. Application kabhi error screen nahi dikhati.

Q19: Aiven DB kya hai aur TaskVoice isse kaise connect hota hai?

Answer: Aiven DB ek cloud-managed database hosting service hai. TaskVoice Aiven cloud par hosted MySQL 8.0 instance se TLS/SSL connection through connect hota hai (`jdbc:mysql://taskvoice-db...aivencloud.com:26284/taskvoice?sslMode=REQUIRED`).

Q20: Render Cloud Deployment architecture explain karo.

Answer: Humne project ko Dockerize kiya (`Dockerfile`). Render humare Git repository se Docker image build karta hai (Maven war package build karke Tomcat 10 `webapps/` me daalta hai), Tomcat container run karta hai on dynamic `$PORT` , aur environment secrets map karta hai.

SECTION E: Security, Testing & AI Usage Justification Q&A**Q21: Team ne project banane me AI Agents ka use kaise kiya?**

Answer: College project guidelines me AI agent use karne ki permission thi. Humne Google Gemini / Antigravity Agent ko ek **AI Pair Programmer** ki tarah use kiya — jiss se initial code scaffolding, Dockerfile creation, aur Glassmorphism CSS layouts jaldi bane. Par **system architecture design, database relational schema, Java EE security filter chain, aur core business rules poori team ne khud implement aur test kiye.**

Q22: Team ke har member ka specific role kya tha?**Answer:**

- *Member 1:* Servlet Architecture, Security Filter Chain, Session Management, BCrypt Hashing.
- *Member 2:* Database SQL Schema, DAO Layer, PreparedStatements, Aiven Cloud DBCP2 Connection Pooling.
- *Member 3:* Google Gemini 1.5 Flash REST Client, Speech API integration, Prompt Engineering, Fallback Engine.
- *Member 4:* JSP/JSTL Views, PDF Report Generation (Apache PDFBox), Docker containerization, Render Cloud Deployment.

Q23: Cross-Site Request Forgery (CSRF) attack se kaise bacha gaya hai?

Answer: `CSRFFilter.java` har session ke liye cryptographically random CSRF token generate karta hai. Har POST form ya AJAX request me yeh token send karna mandatory hota hai. Filter verification fail hone par HTTP 403 Forbidden return karta hai.

Q24: Role-Based Access Control (RBAC) kaise enforce hota hai?

Answer: `RoleFilter.java` `/admin/*`, `/manager/*`, aur `/employee/*` URLs ko intercept karta hai. Session se user role read karke verify karta hai. Agar ek `EMPLOYEE` `/admin/dashboard` access karne ki koshish karega, filter usse Servlet tak jane se pehle hi HTTP 403 block kar dega.

Q25: Reports (PDF & CSV) kaise generate hoti hain?

Answer: `ReportServlet.java` Apache PDFBox library use karke `PdfBuilder` se dynamic PDF work statements aur Apache POI library use karke `CsvBuilder` se CSV datasets in-memory generate karke direct `HttpServletResponse.getOutputStream()` me stream karta hai.

Preparation Guide Complete. Apni group ke saath is Hinglish document ko achhe se padhein aur viva crack karein!